

```
In [1]: import os
import tensorflow as tf

# Tắt Log rác, chỉ hiện lỗi quan trọng
os.environ['TF_CPP_MIN_LOG_LEVEL'] = '2'

# Cấu hình GPU để không chiếm hết VRAM ngay lập tức
gpus = tf.config.list_physical_devices('GPU')
if gpus:
    try:
        for gpu in gpus:
            tf.config.experimental.set_memory_growth(gpu, True)
        print(f"✅ Đã cấu hình Memory Growth cho: {len(gpus)} GPU")
    except RuntimeError as e:
        print(e)
```

✅ Đã cấu hình Memory Growth cho: 1 GPU

```
In [2]: import os
import numpy as np
import tensorflow as tf
from tensorflow.keras import layers, models, optimizers, callbacks
import matplotlib.pyplot as plt
from sklearn.metrics import classification_report, confusion_matrix
import seaborn as sns

# Cấu hình đường dẫn (Trở đúng vào folder bạn đã save ở bước tiền xử lý)
DATA_DIR = r"C:\Users\ADMIN\Documents\DANG_KHOA\processed_data_tiny"

# Hàm Load dữ liệu
def load_data(data_dir):
    print("Đang load dữ liệu từ Drive...")
    X_train = np.load(os.path.join(data_dir, 'X_train.npy'))
    y_train = np.load(os.path.join(data_dir, 'y_train.npy'))

    X_val = np.load(os.path.join(data_dir, 'X_val.npy'))
    y_val = np.load(os.path.join(data_dir, 'y_val.npy'))

    X_test = np.load(os.path.join(data_dir, 'X_test.npy'))
    y_test = np.load(os.path.join(data_dir, 'y_test.npy'))

    print(f"Train shape: {X_train.shape} | Labels: {y_train.shape}")
    print(f"Val shape: {X_val.shape} | Labels: {y_val.shape}")
    print(f"Test shape: {X_test.shape} | Labels: {y_test.shape}")

    return X_train, y_train, X_val, y_val, X_test, y_test

X_train, y_train, X_val, y_val, X_test, y_test = load_data(DATA_DIR)
```

Đang load dữ liệu từ Drive...

Train shape: (240757, 2500, 1) | Labels: (240757,)

Val shape: (32678, 2500, 1) | Labels: (32678,)

Test shape: (62205, 2500, 1) | Labels: (62205,)

```

In [3]: import os
import random
import numpy as np
import tensorflow as tf
from tensorflow.keras import layers, models, regularizers

# =====
# 0. THIẾT LẬP REPRODUCIBILITY (Đã khôi phục)
# =====
SEED = 42
os.environ['PYTHONHASHSEED'] = str(SEED)
random.seed(SEED)
np.random.seed(SEED)
tf.random.set_seed(SEED)
# os.environ['TF_DETERMINISTIC_OPS'] = '1'

def learnable_downsample(x, filters, block_name):
    """
    Kỹ thuật Strided Convolution thay thế MaxPooling (theo Bui An Dong [2] và Squee
    - Kernel=3: Đảm bảo "nhìn" thấy thông tin của 3 mẫu liên kề, tránh mất dữ liệu.
    - Stride=2: Giảm kích thước tín hiệu đi một nửa (Downsampling).
    - Learnable: Có trọng số để mạng tự học cách nén dữ liệu tối ưu nhất.
    """
    x = layers.Conv1D(
        filters=filters,
        kernel_size=3,          # Kernel > 1 là bắt buộc khi stride > 1
        strides=2,              # Thực hiện giảm mẫu tại đây
        padding='same',
        use_bias=False,
        kernel_regularizer=regularizers.l2(L2_RATE),
        name=f'{block_name}_downsample'
    )(x)
    x = layers.BatchNormalization()(x)
    x = layers.LeakyReLU(alpha=0.1)(x)
    return x

# =====
# 1. CẤU HÌNH & DATA AUGMENTATION (Đã khôi phục đầy đủ)
# =====
L2_RATE = 1e-3
INPUT_SHAPE = (2500, 1)

def get_augmentation_layer():
    return tf.keras.Sequential([
        layers.GaussianNoise(0.05),
        layers.Lambda(lambda x: x * tf.random.uniform(tf.shape(x), 0.8, 1.2)),
        layers.Lambda(lambda x: x + tf.random.uniform(tf.shape(x), -0.1, 0.1)), #
        layers.Lambda(lambda x: x + 0.1 * tf.math.sin(
            tf.linspace(0.0, 2.0 * 3.14, tf.shape(x)[1]) * tf.random.uniform([], 0.
        ))[tf.newaxis, :, tf.newaxis])), # sửa chiều broadcasting
        layers.Dropout(0.1)
    ], name="Augmentation_Block")

# =====

```

```

# 2: SE BLOCK (BỔ SUNG PHẦN THIẾU)
# =====
def se_block(x, filters, ratio=4):
    se = layers.GlobalAveragePooling1D()(x)
    se = layers.Dense(filters // ratio, use_bias=False, kernel_regularizer=regularizer)
    se = layers.LeakyReLU(alpha=0.1)(se)
    se = layers.Dense(filters, activation='sigmoid', use_bias=False, kernel_regularizer=regularizer)
    se = layers.Reshape((1, filters))(se)
    x = layers.Multiply()(x, se)
    return x

# =====
# 3. DILATED FIRE MODULE (ĐÃ SỬA LỖI XUNG ĐỘT STRIDE/DILATION)
# =====
def dilated_fire_module(x, squeeze_filters, expand_filters, dilation_rate, block_name):

    s_id = f'fire_{block_name}_squeeze'
    e_id = f'fire_{block_name}_expand'

    # --- INPUT SAVE FOR RESIDUAL ---
    shortcut = x

    # --- 1. SQUEEZE LAYER ---

    squeezed = layers.Conv1D(
        filters=squeeze_filters,
        kernel_size=1,

        padding='same',
        strides=1,
        kernel_regularizer=regularizers.l2(L2_RATE),
        use_bias=False,
        name=s_id
    )(x)
    squeezed = layers.BatchNormalization()(squeezed)
    squeezed = layers.LeakyReLU(alpha=0.1)(squeezed) # <--- THÊM VÀO ĐÂY

    # --- 2. EXPAND LAYER ---

    # Nhánh 1x1
    expand1x1 = layers.Conv1D(
        filters=expand_filters,
        kernel_size=1,

        padding='same',
        strides=1,
        kernel_regularizer=regularizers.l2(L2_RATE),
        use_bias=False,
        name=f'{e_id}_1x1'
    )(squeezed)
    expand1x1 = layers.LeakyReLU(alpha=0.1)(expand1x1) # <--- THÊM VÀO ĐÂY

    # Nhánh 3x3 (Có Dilation)
    # Vì stride=1 nên kết hợp được với dilation_rate > 1
    expand3x3 = layers.Conv1D(
        filters=expand_filters,

```

```

        kernel_size=3,

        padding='same',
        strides=1,
        dilation_rate=dilation_rate,
        kernel_regularizer=regularizers.l2(L2_RATE),
        use_bias=False,
        name=f'{e_id}_3x3_dil{dilation_rate}'
    )(squeezed)
    expand3x3 = layers.LeakyReLU(alpha=0.1)(expand3x3) # <--- THÊM VÀO ĐÂY

# Ghép nối
main_out = layers.Concatenate(axis=-1)([expand1x1, expand3x3])
main_out = layers.BatchNormalization()(main_out)

# SE BLOCK
current_filters = expand_filters * 2
main_out = se_block(main_out, current_filters, ratio=4)

# --- 3. RESIDUAL CONNECTION ---
# Kiểm tra khớp kích thước channels để cộng
if x.shape[-1] == main_out.shape[-1]:
    # Cộng trực tiếp nếu khớp
    x = layers.Add(name=f'fire_{block_name}_add')([main_out, x])
else:
    # Chiếu (Projection) nếu lệch kênh (dùng kernel 1x1, stride 1)
    shortcut = layers.Conv1D(
        filters=main_out.shape[-1],
        kernel_size=1,
        strides=1,
        padding='same',
        use_bias=False,
        name=f'fire_{block_name}_sc_proj'
    )(x)
    shortcut = layers.BatchNormalization()(shortcut)
    x = layers.Add(name=f'fire_{block_name}_add')([main_out, shortcut])

x = layers.LeakyReLU(alpha=0.1)(x) # <--- THAY BẰNG DÒNG MỚI
x = layers.SpatialDropout1D(0.4)(x)

return x

# =====
# 4. MÔ HÌNH CHÍNH
# =====
def build_dilated_se_firenet(input_shape=INPUT_SHAPE):
    inputs = layers.Input(shape=input_shape, name="Input_ECG")

    # Augmentation
    x = get_augmentation_layer()(inputs)

    # --- STEM BLOCK ---
    # Giữ stride=2 để giảm mẫu ngay từ đầu
    x = layers.Conv1D(32, 7, strides=2, padding='same',
        kernel_regularizer=regularizers.l2(L2_RATE),
        use_bias=False, name="Stem_Conv")(x)

```

```

x = layers.BatchNormalization()(x)
x = layers.LeakyReLU(alpha=0.1)(x) # <--- THAY BẰNG DÒNG MỚI

# --- BLOCK 1: Dilation=1 ---
x = dilated_fire_module(x, squeeze_filters=16, expand_filters=16,
                        dilation_rate=1, block_name="block1")
x = learnable_downsample(x, filters=48, block_name="down1")

# --- BLOCK 2: Dilation=2 ---
x = dilated_fire_module(x, squeeze_filters=24, expand_filters=24,
                        dilation_rate=2, block_name="block2")
x = learnable_downsample(x, filters=64, block_name="down2")

# --- BLOCK 3: Dilation=4 ---
x = dilated_fire_module(x, squeeze_filters=32, expand_filters=32,
                        dilation_rate=4, block_name="block3")
x = learnable_downsample(x, filters=96, block_name="down3")

# --- BLOCK 4: Dilation=8 (Deep Feature) ---
x = dilated_fire_module(x, squeeze_filters=48, expand_filters=48,
                        dilation_rate=8, block_name="block4")

# Classifier
# x = layers.GlobalAveragePooling1D(name="Global_Avg_Pool")(x)
gap = layers.GlobalAveragePooling1D(name="Global_Avg_Pool")(x) # Thêm cái này
gmp = layers.GlobalMaxPooling1D(name="Global_Max_Pool")(x) # Thêm cái này
x = layers.Concatenate()([gap, gmp]) # Thêm cái này

x = layers.Dropout(0.5)(x)
outputs = layers.Dense(2, activation='softmax', dtype='float32', name="Output_P

return models.Model(inputs=inputs, outputs=outputs, name="Dilated_SE_FireNet_Co

# Kiểm tra lại số lượng tham số
model = build_dilated_se_firenet()
model.summary()

```

Model: "Dilated_SE_FireNet_Compact"

Layer (type)	Output Shape	Param #	Connected to
=====			
Input_ECG (InputLayer)	[(None, 2500, 1)]	0	[]
Augmentation_Block (Sequential)	(None, 2500, 1)	0	['Input_ECG[0][0]']
Stem_Conv (Conv1D)	(None, 1250, 32)	224	['Augmentation_Bloc k[0][0]']
batch_normalization (BatchNorm alization)	(None, 1250, 32)	128	['Stem_Conv[0][0]']
leaky_re_lu (LeakyReLU)	(None, 1250, 32)	0	['batch_normalizati on[0][0]']
fire_block1_squeeze (Conv1D)	(None, 1250, 16)	512	['leaky_re_lu[0] [0]']
batch_normalization_1 (BatchNo rmalization)	(None, 1250, 16)	64	['fire_block1_squee ze[0][0]']
leaky_re_lu_1 (LeakyReLU)	(None, 1250, 16)	0	['batch_normalizati on_1[0][0]']
fire_block1_expand_1x1 (Conv1D)	(None, 1250, 16)	256	['leaky_re_lu_1[0] [0]']
fire_block1_expand_3x3_dil1 (C onv1D)	(None, 1250, 16)	768	['leaky_re_lu_1[0] [0]']
leaky_re_lu_2 (LeakyReLU)	(None, 1250, 16)	0	['fire_block1_expan d_1x1[0][0]']
leaky_re_lu_3 (LeakyReLU)	(None, 1250, 16)	0	['fire_block1_expan d_3x3_dil1[0][0]']
concatenate (Concatenate)	(None, 1250, 32)	0	['leaky_re_lu_2[0] [0]', 'leaky_re_lu_3[0] [0]']
batch_normalization_2 (BatchNo rmalization)	(None, 1250, 32)	128	['concatenate[0] [0]']
global_average_pooling1d (Glob on_2[0][0]']	(None, 32)	0	['batch_normalizati on_2[0][0]']

alAveragePooling1D)			
dense (Dense) oling1d[0][0]'	(None, 8)	256	['global_average_po ling1d[0][0]']
leaky_re_lu_4 (LeakyReLU)	(None, 8)	0	['dense[0][0]']
dense_1 (Dense) [0]']	(None, 32)	256	['leaky_re_lu_4[0] [0]']
reshape (Reshape)	(None, 1, 32)	0	['dense_1[0][0]']
multiply (Multiply) on_2[0][0]',	(None, 1250, 32)	0	['batch_normalizati on_2[0][0]', 'reshape[0][0]']
fire_block1_add (Add) [0]']	(None, 1250, 32)	0	['multiply[0][0]', 'leaky_re_lu[0] [0]']
leaky_re_lu_5 (LeakyReLU) [0][0]']	(None, 1250, 32)	0	['fire_block1_add [0][0]']
spatial_dropout1d (SpatialDrop out1D)	(None, 1250, 32)	0	['leaky_re_lu_5[0] [0]']
down1_downsample (Conv1D) [0][0]']	(None, 625, 48)	4608	['spatial_dropout1d [0][0]']
batch_normalization_3 (BatchNo [0][0]'] rmalization)	(None, 625, 48)	192	['down1_downsample [0][0]']
leaky_re_lu_6 (LeakyReLU) on_3[0][0]']	(None, 625, 48)	0	['batch_normalizati on_3[0][0]']
fire_block2_squeeze (Conv1D) [0]']	(None, 625, 24)	1152	['leaky_re_lu_6[0] [0]']
batch_normalization_4 (BatchNo ze[0][0]'] rmalization)	(None, 625, 24)	96	['fire_block2_squee ze[0][0]']
leaky_re_lu_7 (LeakyReLU) on_4[0][0]']	(None, 625, 24)	0	['batch_normalizati on_4[0][0]']
fire_block2_expand_1x1 (Conv1D [0]'])	(None, 625, 24)	576	['leaky_re_lu_7[0] [0]']
fire_block2_expand_3x3_dil2 (C [0]'] onv1D)	(None, 625, 24)	1728	['leaky_re_lu_7[0] [0]']

leaky_re_lu_8 (LeakyReLU) d_1x1[0][0]']	(None, 625, 24)	0	['fire_block2_exp
leaky_re_lu_9 (LeakyReLU) d_3x3_dil2[0][	(None, 625, 24)	0	['fire_block2_exp 0]']
concatenate_1 (Concatenate) [0]', [0]']	(None, 625, 48)	0	['leaky_re_lu_8[0] leaky_re_lu_9[0]
batch_normalization_5 (BatchNo [0]'] rmalization)	(None, 625, 48)	192	['concatenate_1[0]
global_average_pooling1d_1 (Gl on_5[0][0]'] obalAveragePooling1D)	(None, 48)	0	['batch_normalizati
dense_2 (Dense) oling1d_1[0][0	(None, 12)	576	['global_average_po ']']
leaky_re_lu_10 (LeakyReLU)	(None, 12)	0	['dense_2[0][0]']
dense_3 (Dense) [0]']	(None, 48)	576	['leaky_re_lu_10[0]
reshape_1 (Reshape)	(None, 1, 48)	0	['dense_3[0][0]']
multiply_1 (Multiply) on_5[0][0]',	(None, 625, 48)	0	['batch_normalizati reshape_1[0][0]']
fire_block2_add (Add) [0]', [0]']	(None, 625, 48)	0	['multiply_1[0] leaky_re_lu_6[0]
leaky_re_lu_11 (LeakyReLU) [0][0]']	(None, 625, 48)	0	['fire_block2_add
spatial_dropout1d_1 (SpatialDr [0]'] opout1D)	(None, 625, 48)	0	['leaky_re_lu_11[0]
down2_downsample (Conv1D) _1[0][0]']	(None, 313, 64)	9216	['spatial_dropout1d
batch_normalization_6 (BatchNo [0][0]'] rmalization)	(None, 313, 64)	256	['down2_downsample
leaky_re_lu_12 (LeakyReLU) on_6[0][0]']	(None, 313, 64)	0	['batch_normalizati

fire_block3_squeeze (Conv1D)	(None, 313, 32)	2048	['leaky_re_lu_12[0][0]']
batch_normalization_7 (Batch Normalization)	(None, 313, 32)	128	['fire_block3_squeeze[0][0]']
leaky_re_lu_13 (LeakyReLU)	(None, 313, 32)	0	['batch_normalization_7[0][0]']
fire_block3_expand_1x1 (Conv1D)	(None, 313, 32)	1024	['leaky_re_lu_13[0][0]']
fire_block3_expand_3x3_dil4 (Conv1D)	(None, 313, 32)	3072	['leaky_re_lu_13[0][0]']
leaky_re_lu_14 (LeakyReLU)	(None, 313, 32)	0	['fire_block3_expand_1x1[0][0]']
leaky_re_lu_15 (LeakyReLU)	(None, 313, 32)	0	['fire_block3_expand_3x3_dil4[0][0]']
concatenate_2 (Concatenate)	(None, 313, 64)	0	['leaky_re_lu_14[0][0]', 'leaky_re_lu_15[0][0]']
batch_normalization_8 (Batch Normalization)	(None, 313, 64)	256	['concatenate_2[0][0]']
global_average_pooling1d_2 (GlobalAveragePooling1D)	(None, 64)	0	['batch_normalization_8[0][0]']
dense_4 (Dense)	(None, 16)	1024	['global_average_pooling1d_2[0][0]']
leaky_re_lu_16 (LeakyReLU)	(None, 16)	0	['dense_4[0][0]']
dense_5 (Dense)	(None, 64)	1024	['leaky_re_lu_16[0][0]']
reshape_2 (Reshape)	(None, 1, 64)	0	['dense_5[0][0]']
multiply_2 (Multiply)	(None, 313, 64)	0	['batch_normalization_8[0][0]', 'reshape_2[0][0]']
fire_block3_add (Add)	(None, 313, 64)	0	['multiply_2[0][0]', 'fire_block3_squeeze[0][0]']

			'leaky_re_lu_12[0]
[0]']			
leaky_re_lu_17 (LeakyReLU)	(None, 313, 64)	0	['fire_block3_add
[0][0]']			
spatial_dropout1d_2 (SpatialDr	(None, 313, 64)	0	['leaky_re_lu_17[0]
[0]']			
opout1D)			
down3_downsample (Conv1D)	(None, 157, 96)	18432	['spatial_dropout1d
_2[0][0]']			
batch_normalization_9 (BatchNo	(None, 157, 96)	384	['down3_downsample
[0][0]']			
rmalization)			
leaky_re_lu_18 (LeakyReLU)	(None, 157, 96)	0	['batch_normalizati
on_9[0][0]']			
fire_block4_squeeze (Conv1D)	(None, 157, 48)	4608	['leaky_re_lu_18[0]
[0]']			
batch_normalization_10 (BatchN	(None, 157, 48)	192	['fire_block4_squee
ze[0][0]']			
ormalization)			
leaky_re_lu_19 (LeakyReLU)	(None, 157, 48)	0	['batch_normalizati
on_10[0][0]']			
fire_block4_expand_1x1 (Conv1D	(None, 157, 48)	2304	['leaky_re_lu_19[0]
[0]']			
)			
fire_block4_expand_3x3_dil8 (C	(None, 157, 48)	6912	['leaky_re_lu_19[0]
[0]']			
onv1D)			
leaky_re_lu_20 (LeakyReLU)	(None, 157, 48)	0	['fire_block4_expan
d_1x1[0][0]']			
leaky_re_lu_21 (LeakyReLU)	(None, 157, 48)	0	['fire_block4_expan
d_3x3_dil8[0][			
			0]']
concatenate_3 (Concatenate)	(None, 157, 96)	0	['leaky_re_lu_20[0]
[0]',			
			'leaky_re_lu_21[0]
[0]']			
batch_normalization_11 (BatchN	(None, 157, 96)	384	['concatenate_3[0]
[0]']			
ormalization)			
global_average_pooling1d_3 (Gl	(None, 96)	0	['batch_normalizati
on_11[0][0]']			

obalAveragePooling1D)			
dense_6 (Dense)	(None, 24)	2304	['global_average_po oling1d_3[0][0]
leaky_re_lu_22 (LeakyReLU)	(None, 24)	0	['dense_6[0][0]']
dense_7 (Dense)	(None, 96)	2304	['leaky_re_lu_22[0] [0]']
reshape_3 (Reshape)	(None, 1, 96)	0	['dense_7[0][0]']
multiply_3 (Multiply)	(None, 157, 96)	0	['batch_normalizati on_11[0][0]', 'reshape_3[0][0]']
fire_block4_add (Add)	(None, 157, 96)	0	['multiply_3[0] [0]', 'leaky_re_lu_18[0] [0]']
leaky_re_lu_23 (LeakyReLU)	(None, 157, 96)	0	['fire_block4_add [0][0]']
spatial_dropout1d_3 (SpatialDr opout1D)	(None, 157, 96)	0	['leaky_re_lu_23[0] [0]']
Global_Avg_Pool (GlobalAverage _3[0][0]') Pooling1D)	(None, 96)	0	['spatial_dropout1d _3[0][0]']
Global_Max_Pool (GlobalMaxPool _3[0][0]') ing1D)	(None, 96)	0	['spatial_dropout1d _3[0][0]']
concatenate_4 (Concatenate)	(None, 192)	0	['Global_Avg_Pool [0][0]', 'Global_Max_Pool [0][0]']
dropout_1 (Dropout)	(None, 192)	0	['concatenate_4[0] [0]']
Output_Prob (Dense)	(None, 2)	386	['dropout_1[0][0]']

```
=====
=====
Total params: 68,546
Trainable params: 67,346
Non-trainable params: 1,200
```

```

In [4]: from sklearn.metrics import classification_report, accuracy_score, f1_score
import pandas as pd
from sklearn.utils import class_weight
import tensorflow as tf
import tensorflow.keras.backend as K

# =====
# 4. CHUẨN BỊ VÀ HUẤN LUYỆN
# =====

...

def categorical_focal_loss(gamma=2.0):
    """
    Focal Loss theo chuẩn nghiên cứu của Petmezas et al. (2021) cho ECG.
    - gamma=2.0: Giúp model tập trung vào các mẫu khó (AFIB).
    - Không dùng alpha: Để tránh xung đột với class_weight trong model.fit().
    """
    def focal_loss_fixed(y_true, y_pred):
        # 1. Clip giá trị để tránh log(0) gây lỗi NaN
        epsilon = K.epsilon()
        y_pred = K.clip(y_pred, epsilon, 1.0 - epsilon)

        # 2. Tính Cross Entropy cơ bản
        cross_entropy = -y_true * K.log(y_pred)

        # 3. Tính hệ số Focusing  $(1 - p)^{\gamma}$ 
        # Mẫu nào model càng tự tin (p cao) thì loss càng bị giảm đi
        weight = K.pow((1 - y_pred), gamma)

        # 4. Tổng hợp
        loss = weight * cross_entropy

        # 5. Tính tổng loss
        return K.sum(loss, axis=1)

    return focal_loss_fixed
...

...

# A. One-hot encoding
y_train_onehot = tf.keras.utils.to_categorical(y_train, num_classes=2)
y_val_onehot = tf.keras.utils.to_categorical(y_val, num_classes=2)

# B. Class Weights (Giữ nguyên chiến thuật đã thành công)
y_integers = np.argmax(y_train_onehot, axis=1)
class_weights = class_weight.compute_class_weight(
    class_weight='balanced',
    classes=np.unique(y_integers),
    y=y_integers
)
class_weight_dict = dict(enumerate(class_weights))

print(f"📊 Trọng số lớp (Fixed Seed): {class_weight_dict}")
...

```

```

model = build_dilated_se_firenet()

# 1. Label Smoothing Loss: Chìa khóa chống Loss nhảy
# Thay vì phạt Lỗi dựa trên nhãn cứng (0, 1), ta dùng nhãn mềm (0.05, 0.95)
# Lưu ý: Khi dùng Label Smoothing, ta cần dùng Categorical Crossentropy (One-hot)
# thay vì Sparse. Vì Sparse không hỗ trợ smoothing trực tiếp trong một số bản TF cũ
loss_fn = tf.keras.losses.CategoricalCrossentropy(label_smoothing=0.1)

# 2. Chuyển đổi nhãn sang One-hot (Bắt buộc cho Label Smoothing)
y_train_onehot = tf.keras.utils.to_categorical(y_train, num_classes=2)
y_val_onehot    = tf.keras.utils.to_categorical(y_val, num_classes=2)

# 3. Optimizer với Cosine Decay (Giảm LR mượt mà)
# Tổng step = (240757 / 128) * 60 epochs ~= 112,800 steps
initial_lr = 1e-4
lr_schedule = tf.keras.optimizers.schedules.CosineDecay(
    initial_learning_rate=initial_lr,
    decay_steps=112800, # Ước lượng số steps cho 60 epochs
    alpha=0.01         # LR cuối cùng = 1% LR ban đầu
)

# C. Optimizer & Callbacks
# optimizer = optimizers.Adam(learning_rate=0.0001)
optimizer = tf.keras.optimizers.Adam(learning_rate=lr_schedule)

# --- THÊM CÁC METRICS QUAN TRỌNG CHO Y SINH ---
...
metrics_list = [
    'accuracy',
    tf.keras.metrics.AUC(name='auc'),          # Quan trọng nhất
    tf.keras.metrics.Recall(name='sensitivity'), # Độ nhạy (Quan trọng nhì: Không b
    tf.keras.metrics.Precision(name='precision') # Độ chính xác (Tránh báo động giả
]
...

metrics_list = ['accuracy', tf.keras.metrics.AUC(name='auc')]

model.compile(
    optimizer=optimizer,
    # loss=categorical_focal_loss(gamma=2.0),
    loss = loss_fn,
    metrics=metrics_list
)

callbacks_list = [
    # Chỉ lưu model tốt nhất
    callbacks.ModelCheckpoint('best_robust_model_final.keras', monitor='val_loss',
    # Dừng sớm nếu không cải thiện trong 15 epochs
    callbacks.EarlyStopping(monitor='val_loss', patience=10, restore_best_weights=T
]

print(f"🚀 Bắt đầu huấn luyện (Seed={SEED})...")
...
history = model.fit(

```

```

X_train, y_train_onehot,
validation_data=(X_val, y_val_onehot),
epochs=60,
batch_size=128,
callbacks=callbacks_list,
class_weight=class_weight_dict,
shuffle=True # Shuffle này cũng sẽ được cố định bởi np.random.seed
)
'''

history = model.fit(
    X_train, y_train_onehot, # Dùng One-hot
    validation_data=(X_val, y_val_onehot),
    epochs=60,
    batch_size=128,
    callbacks=callbacks_list,
    shuffle=True,
    verbose=1
)

```

🚀 Bắt đầu huấn luyện (Seed=42)...

Epoch 1/60

1881/1881 [=====] - ETA: 0s - loss: 2.0456 - accuracy: 0.5512 - auc: 0.5621

Epoch 1: val_loss improved from inf to 1.27654, saving model to best_robust_model_final.keras

1881/1881 [=====] - 284s 50ms/step - loss: 2.0456 - accuracy: 0.5512 - auc: 0.5621 - val_loss: 1.2765 - val_accuracy: 0.7977 - val_auc: 0.8916

Epoch 2/60

1880/1881 [=====>.] - ETA: 0s - loss: 1.1449 - accuracy: 0.7377 - auc: 0.8251

Epoch 2: val_loss improved from 1.27654 to 0.92831, saving model to best_robust_model_final.keras

1881/1881 [=====] - 94s 50ms/step - loss: 1.1449 - accuracy: 0.7378 - auc: 0.8251 - val_loss: 0.9283 - val_accuracy: 0.8925 - val_auc: 0.9656

Epoch 3/60

1880/1881 [=====>.] - ETA: 0s - loss: 0.8269 - accuracy: 0.8930 - auc: 0.9605

Epoch 3: val_loss improved from 0.92831 to 0.70360, saving model to best_robust_model_final.keras

1881/1881 [=====] - 94s 50ms/step - loss: 0.8268 - accuracy: 0.8930 - auc: 0.9605 - val_loss: 0.7036 - val_accuracy: 0.9195 - val_auc: 0.9745

Epoch 4/60

1881/1881 [=====] - ETA: 0s - loss: 0.6053 - accuracy: 0.9506 - auc: 0.9871

Epoch 4: val_loss improved from 0.70360 to 0.56138, saving model to best_robust_model_final.keras

1881/1881 [=====] - 94s 50ms/step - loss: 0.6053 - accuracy: 0.9506 - auc: 0.9871 - val_loss: 0.5614 - val_accuracy: 0.9186 - val_auc: 0.9747

Epoch 5/60

1881/1881 [=====] - ETA: 0s - loss: 0.4569 - accuracy: 0.9683 - auc: 0.9940

Epoch 5: val_loss improved from 0.56138 to 0.45387, saving model to best_robust_model_final.keras

1881/1881 [=====] - 94s 50ms/step - loss: 0.4569 - accuracy: 0.9683 - auc: 0.9940 - val_loss: 0.4539 - val_accuracy: 0.9165 - val_auc: 0.9812

Epoch 6/60

1880/1881 [=====>.] - ETA: 0s - loss: 0.3685 - accuracy: 0.9807 - auc: 0.9974

Epoch 6: val_loss improved from 0.45387 to 0.34653, saving model to best_robust_model_final.keras

1881/1881 [=====] - 95s 50ms/step - loss: 0.3684 - accuracy: 0.9807 - auc: 0.9974 - val_loss: 0.3465 - val_accuracy: 0.9732 - val_auc: 0.9955

Epoch 7/60

1880/1881 [=====>.] - ETA: 0s - loss: 0.3156 - accuracy: 0.9898 - auc: 0.9987

Epoch 7: val_loss improved from 0.34653 to 0.29653, saving model to best_robust_model_final.keras

1881/1881 [=====] - 95s 50ms/step - loss: 0.3156 - accuracy: 0.9898 - auc: 0.9987 - val_loss: 0.2965 - val_accuracy: 0.9863 - val_auc: 0.9964

Epoch 8/60

1880/1881 [=====>.] - ETA: 0s - loss: 0.2859 - accuracy: 0.9931 - auc: 0.9990

Epoch 8: val_loss improved from 0.29653 to 0.28547, saving model to best_robust_model_final.keras

1881/1881 [=====] - 94s 50ms/step - loss: 0.2859 - accuracy:

y: 0.9931 - auc: 0.9990 - val_loss: 0.2855 - val_accuracy: 0.9869 - val_auc: 0.9968
Epoch 9/60
1880/1881 [=====>.] - ETA: 0s - loss: 0.2682 - accuracy: 0.9944 - auc: 0.9993
Epoch 9: val_loss improved from 0.28547 to 0.26201, saving model to best_robust_model_final.keras
1881/1881 [=====] - 94s 50ms/step - loss: 0.2682 - accuracy: 0.9944 - auc: 0.9993 - val_loss: 0.2620 - val_accuracy: 0.9916 - val_auc: 0.9962
Epoch 10/60
1880/1881 [=====>.] - ETA: 0s - loss: 0.2574 - accuracy: 0.9947 - auc: 0.9993
Epoch 10: val_loss did not improve from 0.26201
1881/1881 [=====] - 95s 50ms/step - loss: 0.2574 - accuracy: 0.9947 - auc: 0.9993 - val_loss: 0.2858 - val_accuracy: 0.9775 - val_auc: 0.9955
Epoch 11/60
1880/1881 [=====>.] - ETA: 0s - loss: 0.2503 - accuracy: 0.9952 - auc: 0.9993
Epoch 11: val_loss did not improve from 0.26201
1881/1881 [=====] - 95s 50ms/step - loss: 0.2503 - accuracy: 0.9952 - auc: 0.9993 - val_loss: 0.2994 - val_accuracy: 0.9657 - val_auc: 0.9935
Epoch 12/60
1881/1881 [=====] - ETA: 0s - loss: 0.2447 - accuracy: 0.9955 - auc: 0.9994
Epoch 12: val_loss did not improve from 0.26201
1881/1881 [=====] - 95s 51ms/step - loss: 0.2447 - accuracy: 0.9955 - auc: 0.9994 - val_loss: 0.2754 - val_accuracy: 0.9824 - val_auc: 0.9959
Epoch 13/60
1880/1881 [=====>.] - ETA: 0s - loss: 0.2411 - accuracy: 0.9956 - auc: 0.9994
Epoch 13: val_loss improved from 0.26201 to 0.25067, saving model to best_robust_model_final.keras
1881/1881 [=====] - 95s 50ms/step - loss: 0.2411 - accuracy: 0.9956 - auc: 0.9994 - val_loss: 0.2507 - val_accuracy: 0.9923 - val_auc: 0.9968
Epoch 14/60
1880/1881 [=====>.] - ETA: 0s - loss: 0.2379 - accuracy: 0.9958 - auc: 0.9994
Epoch 14: val_loss did not improve from 0.25067
1881/1881 [=====] - 95s 50ms/step - loss: 0.2379 - accuracy: 0.9958 - auc: 0.9994 - val_loss: 0.2650 - val_accuracy: 0.9866 - val_auc: 0.9962
Epoch 15/60
1880/1881 [=====>.] - ETA: 0s - loss: 0.2355 - accuracy: 0.9959 - auc: 0.9994
Epoch 15: val_loss did not improve from 0.25067
1881/1881 [=====] - 95s 51ms/step - loss: 0.2355 - accuracy: 0.9959 - auc: 0.9994 - val_loss: 0.2691 - val_accuracy: 0.9831 - val_auc: 0.9962
Epoch 16/60
1881/1881 [=====] - ETA: 0s - loss: 0.2335 - accuracy: 0.9962 - auc: 0.9995
Epoch 16: val_loss did not improve from 0.25067
1881/1881 [=====] - 95s 50ms/step - loss: 0.2335 - accuracy: 0.9962 - auc: 0.9995 - val_loss: 0.3216 - val_accuracy: 0.9294 - val_auc: 0.9838
Epoch 17/60
1881/1881 [=====] - ETA: 0s - loss: 0.2317 - accuracy: 0.9962 - auc: 0.9996
Epoch 17: val_loss did not improve from 0.25067
1881/1881 [=====] - 95s 50ms/step - loss: 0.2317 - accuracy:


```

y: 0.9962 - auc: 0.9996 - val_loss: 0.2832 - val_accuracy: 0.9693 - val_auc: 0.9943
Epoch 18/60
1880/1881 [=====>.] - ETA: 0s - loss: 0.2304 - accuracy: 0.99
62 - auc: 0.9996
Epoch 18: val_loss did not improve from 0.25067
1881/1881 [=====] - 95s 50ms/step - loss: 0.2304 - accurac
y: 0.9962 - auc: 0.9996 - val_loss: 0.2729 - val_accuracy: 0.9767 - val_auc: 0.9950
Epoch 19/60
1880/1881 [=====>.] - ETA: 0s - loss: 0.2292 - accuracy: 0.99
63 - auc: 0.9996
Epoch 19: val_loss did not improve from 0.25067
1881/1881 [=====] - 95s 51ms/step - loss: 0.2292 - accurac
y: 0.9963 - auc: 0.9996 - val_loss: 0.2800 - val_accuracy: 0.9733 - val_auc: 0.9948
Epoch 20/60
1880/1881 [=====>.] - ETA: 0s - loss: 0.2281 - accuracy: 0.99
62 - auc: 0.9996
Epoch 20: val_loss did not improve from 0.25067
1881/1881 [=====] - 95s 50ms/step - loss: 0.2282 - accurac
y: 0.9962 - auc: 0.9996 - val_loss: 0.3196 - val_accuracy: 0.9293 - val_auc: 0.9836
Epoch 21/60
1880/1881 [=====>.] - ETA: 0s - loss: 0.2271 - accuracy: 0.99
65 - auc: 0.9996
Epoch 21: val_loss did not improve from 0.25067
1881/1881 [=====] - 95s 50ms/step - loss: 0.2271 - accurac
y: 0.9965 - auc: 0.9996 - val_loss: 0.3182 - val_accuracy: 0.9281 - val_auc: 0.9839
Epoch 22/60
1881/1881 [=====] - ETA: 0s - loss: 0.2261 - accuracy: 0.99
65 - auc: 0.9996
Epoch 22: val_loss did not improve from 0.25067
1881/1881 [=====] - 95s 50ms/step - loss: 0.2261 - accurac
y: 0.9965 - auc: 0.9996 - val_loss: 0.3180 - val_accuracy: 0.9261 - val_auc: 0.9834
Epoch 23/60
1881/1881 [=====] - ETA: 0s - loss: 0.2253 - accuracy: 0.99
66 - auc: 0.9996
Epoch 23: val_loss did not improve from 0.25067
Restoring model weights from the end of the best epoch: 13.
1881/1881 [=====] - 95s 50ms/step - loss: 0.2253 - accurac
y: 0.9966 - auc: 0.9996 - val_loss: 0.2746 - val_accuracy: 0.9689 - val_auc: 0.9947
Epoch 23: early stopping

```

```

In [5]: from sklearn.metrics import f1_score, precision_score, recall_score, roc_curve, auc
import time

# =====
# BƯỚC 5: ĐÁNH GIÁ (UPDATED)
# =====

final_model = model

print("Đang dự đoán trên tập Test...")
# Lưu ý: Model dự đoán ra xác suất [p0, p1]
start_time = time.perf_counter()
y_pred_probs = final_model.predict(X_test)
end_time = time.perf_counter()

```

```

inference_time = end_time - start_time
time_per_sample = inference_time / len(X_test)
# Lấy vị trí có xác suất lớn nhất -> Ra nhãn 0 hoặc 1
y_pred = np.argmax(y_pred_probs, axis=1)

# --- BÁO CÁO ---
print("\n--- BÁO CÁO KẾT QUẢ TRÊN TẬP TEST ---")
print(f"\nThời gian dự đoán toàn bộ tập test: {inference_time:.4f} giây")
print(f"Thời gian trung bình mỗi mẫu: {time_per_sample*1000:.4f} ms\n")

# y_test của bạn vẫn là số nguyên (0, 1) nên so sánh trực tiếp được
print(classification_report(y_test, y_pred, target_names=['Normal (0)', 'AFib (1)'])

# --- CONFUSION MATRIX ---
plt.figure(figsize=(6, 5))
cm = confusion_matrix(y_test, y_pred)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=['Normal', 'AFib'],
plt.xlabel('Dự đoán')
plt.ylabel('Thực tế')
plt.title('Confusion Matrix')
plt.show()

# --- BIỂU ĐỒ LOSS & ACCURACY ---
plt.figure(figsize=(12, 4))

plt.subplot(1, 2, 1)
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Val Loss')
plt.title('Loss (có Label Smoothing)')
plt.legend()

plt.subplot(1, 2, 2)
plt.plot(history.history['accuracy'], label='Train Acc')
plt.plot(history.history['val_accuracy'], label='Val Acc')
plt.title('Accuracy')
plt.legend()

# --- ROC CURVE ---
y_pred_prob_afib = y_pred_probs[:, 1] # xác suất Lớp 1
fpr, tpr, thresholds = roc_curve(y_test, y_pred_prob_afib)
roc_auc = auc(fpr, tpr)

plt.figure(figsize=(6, 6))
plt.plot(fpr, tpr, color='blue', lw=2, label=f'ROC curve (AUC = {roc_auc:.4f})')
plt.plot([0, 1], [0, 1], color='gray', lw=1, linestyle='--', label='Random')
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate (FPR)')
plt.ylabel('True Positive Rate (TPR)')
plt.title('ROC Curve - AFib Detection')
plt.legend(loc="lower right")
plt.grid(True)
plt.show()

plt.show()

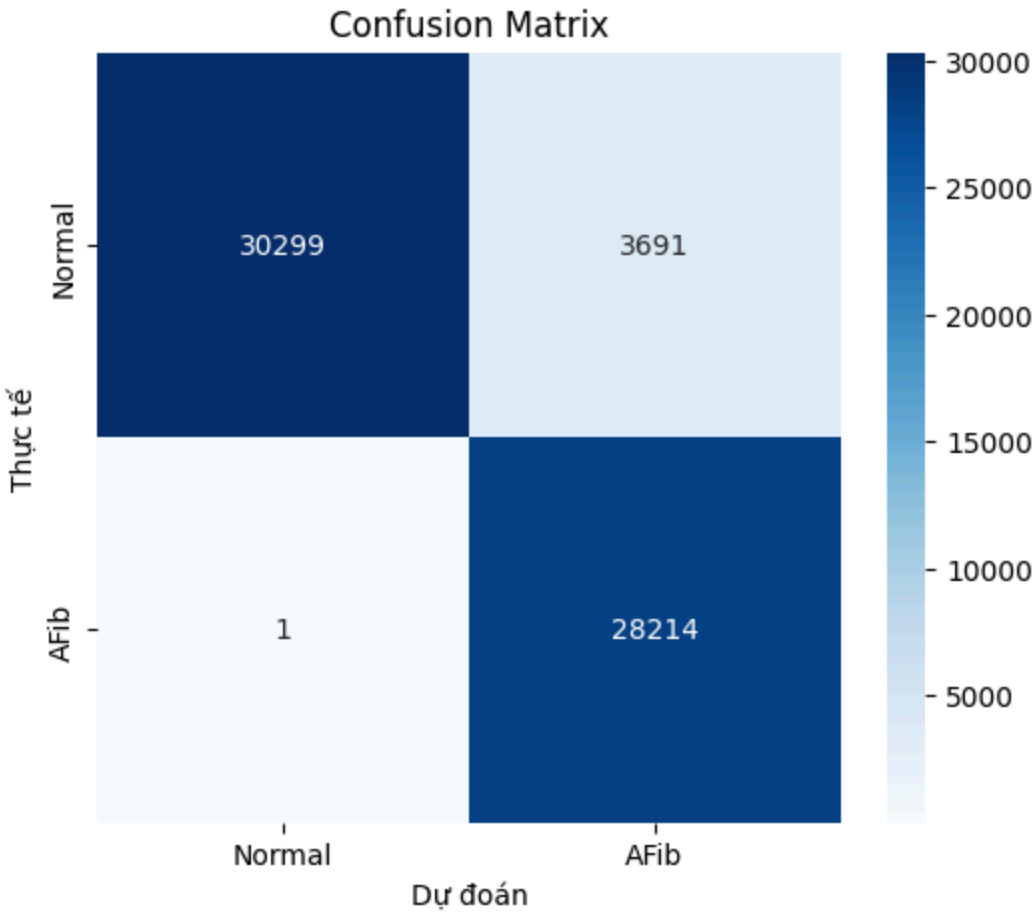
```

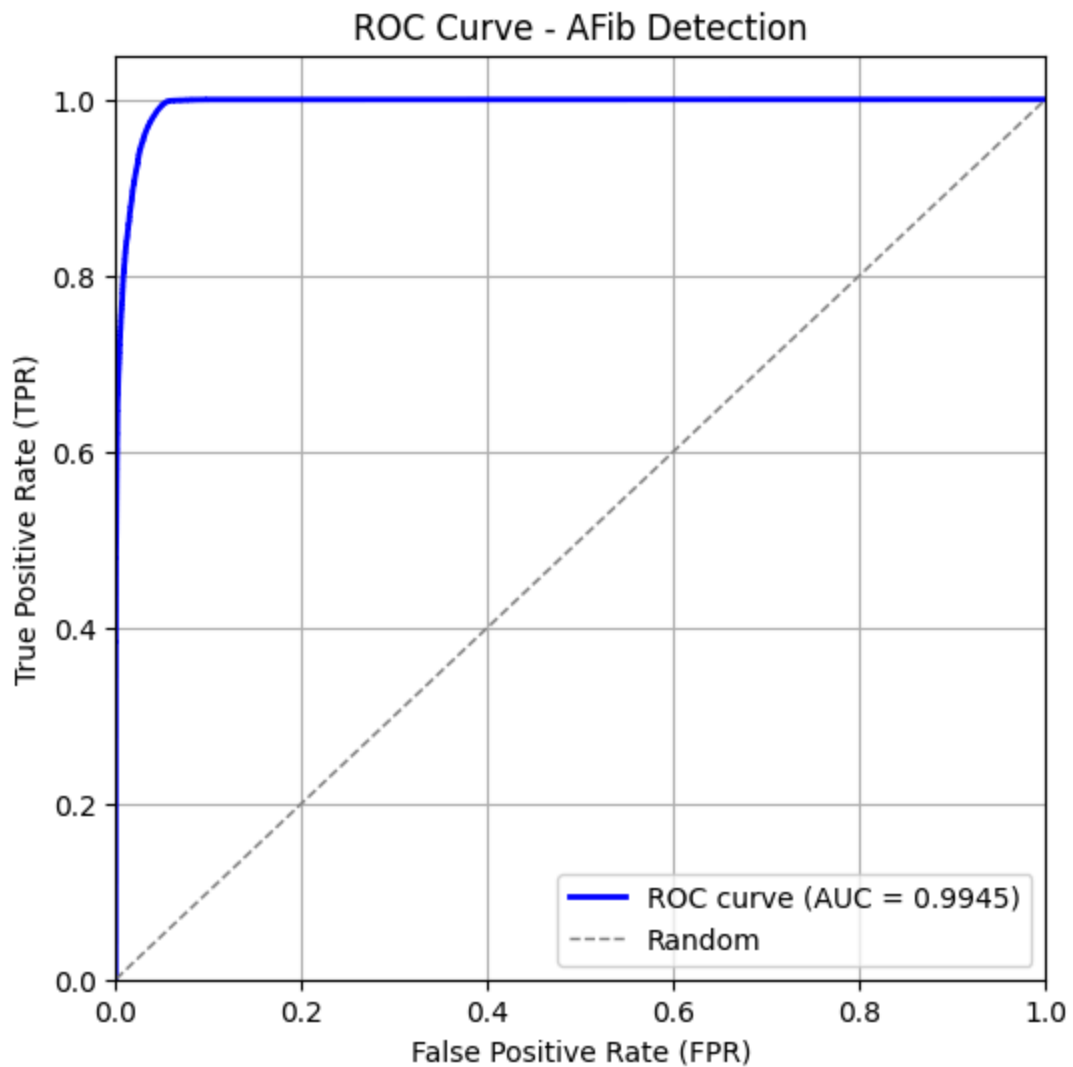
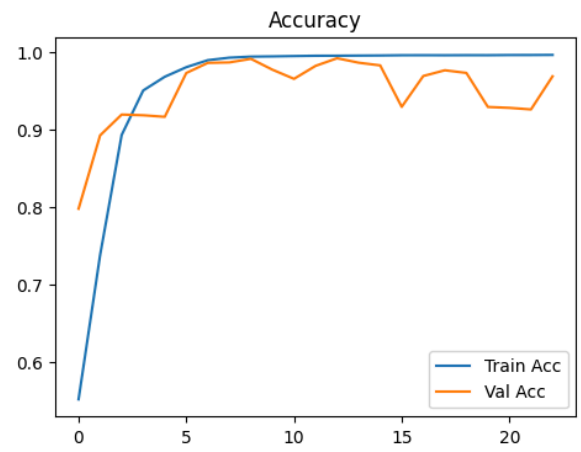
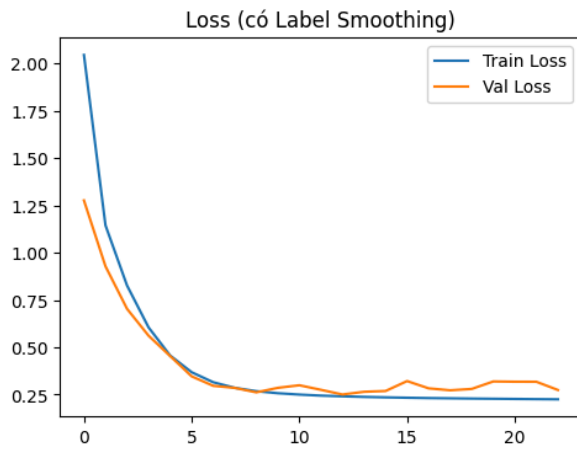
Đang dự đoán trên tập Test...
1944/1944 [=====] - 7s 3ms/step

--- BÁO CÁO KẾT QUẢ TRÊN TẬP TEST ---

Thời gian dự đoán toàn bộ tập test: 7.4074 giây
Thời gian trung bình mỗi mẫu: 0.1191 ms

	precision	recall	f1-score	support
Normal (0)	1.0000	0.8914	0.9426	33990
AFib (1)	0.8843	1.0000	0.9386	28215
accuracy			0.9406	62205
macro avg	0.9421	0.9457	0.9406	62205
weighted avg	0.9475	0.9406	0.9408	62205





```
In [6]: # =====
# BƯỚC 5: ĐÁNH GIÁ TOÀN DIỆN & TỐI ƯU NGUỖNG (SCIENTIFIC REPORT)
# =====
from sklearn.metrics import classification_report, confusion_matrix, roc_curve, auc
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np

# 1. Dự đoán xác suất trên tập VAL và TEST
```

```

print("🕒 Đang dự đoán xác suất...")
# Lấy xác suất Lớp 1 (Rung nhĩ)
y_val_probs = model.predict(X_val, verbose=0)[: , 1]
y_test_probs = model.predict(X_test, verbose=0)[: , 1]

# =====
# A. TÌM NGUỒN TỐI ƯU TRÊN TẬP VAL (Threshold Tuning)
# =====
# Mục tiêu: Giữ Sensitivity (Recall AFIB) > 99% nhưng tăng Precision/Specificity
fpr, tpr, thresholds = roc_curve(y_val, y_val_probs)

# Tìm ngưỡng sao cho Sensitivity >= 0.99
optimal_idx = np.argmax(tpr >= 0.99)
# Tuy nhiên, để an toàn và giảm báo động giả, ta thường dùng Youden's J statistic
# J = Sensitivity + Specificity - 1
J = tpr - fpr
ix = np.argmax(J)
best_thresh = thresholds[ix]

print(f"\n🔍 KẾT QUẢ TỐI ƯU HÓA TRÊN TẬP VAL:")
print(f"    - Ngưỡng tối ưu (Best Threshold): {best_thresh:.4f}")
print(f"    - Tại ngưỡng này trên Val: Sensitivity={tpr[ix]:.4f}, Specificity={1-fpr[ix]:.4f}")

# =====
# B. ÁP DỤNG NGUỒN VÀO TẬP TEST (Đánh giá công bằng)
# =====
# Thay vì dùng argmax (ngưỡng 0.5), ta dùng ngưỡng best_thresh tìm được
y_pred_new = (y_test_probs >= best_thresh).astype(int)

print(f"\n{' '*20} BÁO CÁO KẾT QUẢ CUỐI CÙNG (TẬP TEST) {' '*20}")
print(f"Áp dụng ngưỡng decision threshold = {best_thresh:.4f}")

# Classification Report
print(classification_report(y_test, y_pred_new, target_names=['Normal (0)', 'AFib (1)']))

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred_new)
plt.figure(figsize=(6, 5))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
             xticklabels=['Normal', 'AFib'], yticklabels=['Normal', 'AFib'])
plt.title(f'Confusion Matrix (Threshold={best_thresh:.3f})')
plt.ylabel('Thực tế (Ground Truth)')
plt.xlabel('Dự đoán (Prediction)')
plt.show()

# =====
# C. VẼ ĐỒ THỊ ROC & TRAINING HISTORY
# =====
plt.figure(figsize=(15, 6))

# --- 1. ROC CURVE ---
plt.subplot(1, 2, 1)
# Tính ROC cho Test
fpr_test, tpr_test, _ = roc_curve(y_test, y_test_probs)
roc_auc = auc(fpr_test, tpr_test)

```

```

plt.plot(fpr_test, tpr_test, color='darkorange', lw=2, label=f'ROC curve (AUC = {ro
plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--')
plt.scatter(fpr[ix], tpr[ix], marker='o', color='black', label=f'Optimal Threshold
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate (1 - Specificity)')
plt.ylabel('True Positive Rate (Sensitivity)')
plt.title('Receiver Operating Characteristic (ROC) - Test Set')
plt.legend(loc="lower right")
plt.grid(True, alpha=0.3)

# --- 2. LOSS & ACCURACY ---
plt.subplot(1, 2, 2)
# Vẽ 2 trục y khác nhau: Trái là Loss, Phải là Accuracy
ax1 = plt.gca()
ax2 = ax1.twinx()

ln1 = ax1.plot(history.history['loss'], 'r-', label='Train Loss', alpha=0.6)
ln2 = ax1.plot(history.history['val_loss'], 'r--', label='Val Loss', alpha=0.6)
ln3 = ax2.plot(history.history['accuracy'], 'b-', label='Train Acc', alpha=0.6)
ln4 = ax2.plot(history.history['val_accuracy'], 'b--', label='Val Acc', alpha=0.6)

ax1.set_xlabel('Epoch')
ax1.set_ylabel('Loss', color='r')
ax2.set_ylabel('Accuracy', color='b')
ax1.set_title('Training Dynamics (Loss & Accuracy)')

# Gôm Label cho Legend
lns = ln1+ln2+ln3+ln4
labs = [l.get_label() for l in lns]
ax1.legend(lns, labs, loc=0)
plt.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

```

⌚ Đang dự đoán xác suất...

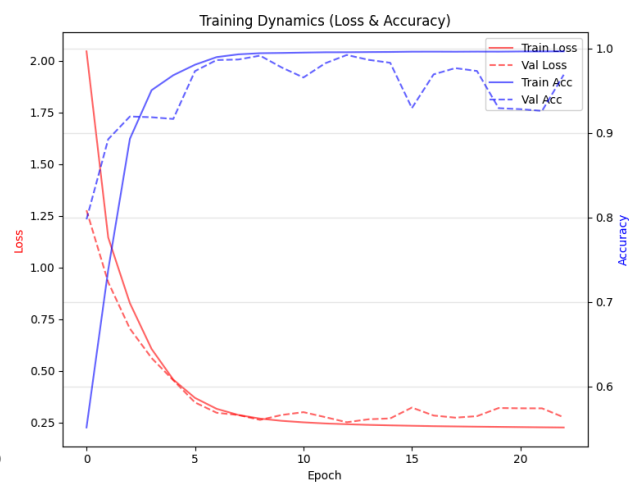
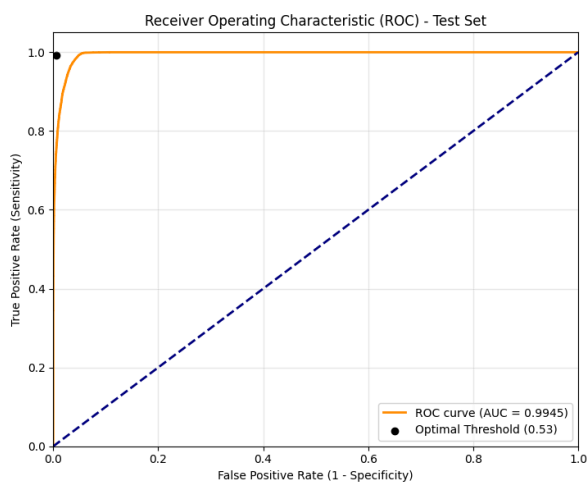
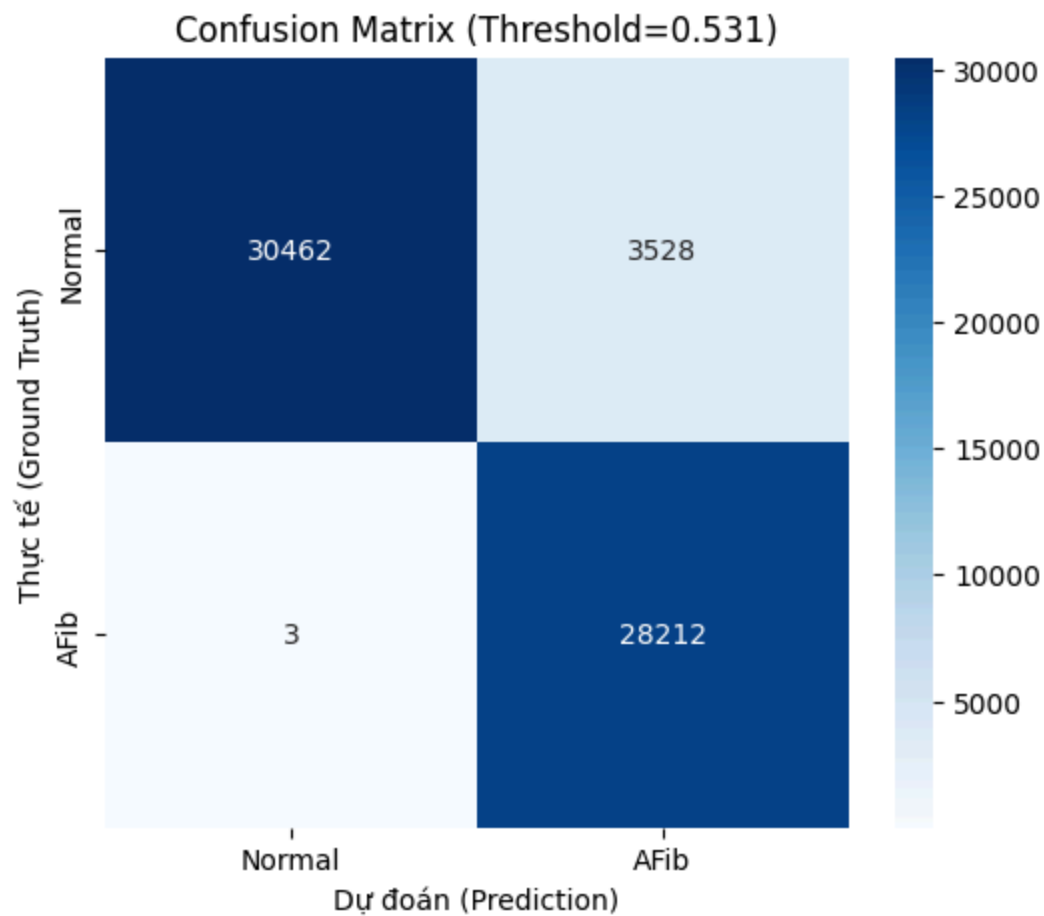
🔍 KẾT QUẢ TỐI ƯU HÓA TRÊN TẬP VAL:

- Ngưỡng tối ưu (Best Threshold): 0.5305
- Tại ngưỡng này trên Val: Sensitivity=0.9914, Specificity=0.9937

===== BÁO CÁO KẾT QUẢ CUỐI CÙNG (TẬP TEST) =====

Áp dụng ngưỡng decision threshold = 0.5305

	precision	recall	f1-score	support
Normal (0)	0.9999	0.8962	0.9452	33990
AFib (1)	0.8888	0.9999	0.9411	28215
accuracy			0.9432	62205
macro avg	0.9444	0.9480	0.9432	62205
weighted avg	0.9495	0.9432	0.9434	62205



In []: